

why you suck at vibe coding (and the comprehensive guide to fix you)

Let's get this out of the way now. Vibe coding isn't the problem. You are. You heard you could talk to AI coding agents and ship software. And you thought you were a magician. So you opened an AI up, described your app idea in a sentence, and expected fucking magic to return. But to surprise, you got broken code, hallucinated UI and colors, pages that don't route or connect, and an app that 'kinda' works until it doesn't really at all.

And you blamed AI for the mistakes. Like an idiot.

Here's the truth: AI doesn't hallucinate because it's broken. It hallucinates because you gave it nothing to hold onto. No structure, clarity, or foundation.

Vibe coding works. But only if you understand what you're building and give your AI agent a true comprehensive system to work within. What follows is everything you need to know: the building blocks, the vocabulary, the real workflows, all of it explained simply enough a caveman could do it.

If you read this and still can't ship, the problem is effort, not information.

This isn't a thread you skim and forget. This is the entire system, start to finish for vibe coding. Bookmark it now. Save it to your Clawdbot's memory. Come back to it every time you start a new project. The people who treat this as a reference manual will build incredible things. The people who skim it will stay stuck and broken.

Now let's fix you.

1: Why You're Failing

You're failing because you skipped the fundamentals.

You don't know what a component is. You don't know what state means. You don't understand why your button doesn't do anything when you click it. You don't know why your site looks good on your laptop but breaks on your phone.

And because you don't know these things, you can't describe them to AI.

AI is a translator. It converts your intent into code. But if your intent is shitty, the code will be shitty. If you can't articulate what you want, AI guesses. And guesses compound into total and pure chaos.

The fix isn't better prompts. The fix is better understanding.

Once you know what you're building, prompting becomes trivial. The words come naturally because you finally know what to ask for.

2: The Documentation-First System

This is where everyone gets it wrong.

You open Cursor, open a chat, start describing your app, and let AI start coding immediately. No plan, no reference, and no sources of truth.

This is why your project falls apart after the first few files begin.

The real system is documentation first, code second. Always do this.

Before you write a single line of code, you should write your project's canonical documentation markdown files. Clear, specific, unambiguous descriptions of what you're building.

Why? Because AI coding tools operate with high capability but low certainty. They execute tasks without structural guardrails. The absence of locked constraints and authoritative documentation causes AI to hallucinate requirements, make unauthorized architectural decisions, and produce code that solves problems you never articulated.

The failure mode is not lack of coding ability. The failure mode is lack of discipline and context preservation.

Here's the documentation stack you should write before touching any code. Six canonical docs that define your entire project, plus some files that keep AI aligned and persistent across sessions.

The Canonical Docs (your knowledge base):

1.

[PRD.md](#)

(Product Requirements Document) - The full spec. What you're building, who it's for, what features exist, what's in scope, and what's explicitly out of scope. User stories, success criteria, non-goals, and every feature with specific criteria. This is your contract. AI reads this and knows exactly what "done" looks like for you. Don't have this? You're not building an app. You're praying for one.

2. APP_FLOW.md - Every page and every user navigation path documented in plain English. What triggers each flow. Step-by-step sequences with decision points, what happens on success, what happens on error, and the screen inventory with routes. This prevents AI from guessing how users move through your app.

3. TECH_STACK.md - Every package, dependency, API, and tool locked to exact versions. No ambiguity. When AI sees "use React," it might pick any version. When it sees "Next.js 14.1.0, React 18.2.0, TypeScript 5.3.3," it builds exactly what you specified. This doc eliminates hallucinated dependencies and random tech choices.

4. FRONTEND_GUIDELINES.md - Your complete design system. Fonts, color palette with exact hex codes, spacing scale, layout rules, component styles, responsive breakpoints, and UI library preferences. Every visual decision locked down. AI references this for every component it creates. No more random colors or inconsistent spacing while you keep vibing.

5. BACKEND_STRUCTURE.md - Database schema with every table, column, type, and relationship defined. Authentication logic, API endpoint contracts, storage rules, and edge cases. If you're using Supabase, this doc contains the exact SQL structure. AI builds your backend against this blueprint, not against its own assumptions.

6. IMPLEMENTATION_PLAN.md - A step-by-step build sequence. Not "build the app." More like: step 1.1 initialize the project, step 1.2 install dependencies from TECH_STACK.md, step 1.3 create folder structure, step 2.1 build the navbar component per FRONTEND_GUIDELINES.md, and so on. The more steps, the less AI guesses. The less AI guesses, the fewer hallucinations.

These docs cross-reference each other. The PRD defines features, the APP FLOW defines how users experience them, the TECH STACK defines what builds them, the FRONTEND GUIDELINES define how they look, the BACKEND STRUCTURE defines how data works, and the IMPLEMENTATION PLAN defines the build order.

This is your knowledge base. AI will read these and have everything it needs.

The Two Session Files (your persistence layer):

CLAUDE.md

- This is the file AI reads first, automatically, every session. It contains the rules, constraints, patterns, and context that every AI session must follow. Your tech stack summary, your file naming conventions. your component patterns, and your design system tokens. It's what's allowed and what's forbidden. Think of it as the AI's operating manual for your specific project. Claude Code can read this from the project root without you even asking.

progress.txt - This is the file everyone misses. This file tracks what's been done, what's in progress, and what's next. Every time you finish a feature, you update this file. Every time you start a new session, every time you open a new terminal window, every time you switch

branches, AI reads this file first for contextual memory of your progress. Without it, every new session starts from zero context and a whole lot of fuckups along the way. With it, AI picks up exactly where you left off.

Here's why this matters: AI has no memory between sessions. When you close a terminal, open a new one, or start a new chat, everything is gone. Progress.txt is your external memory. It's the bridge between sessions.

Update it religiously. After every completed feature you implement, meticulously document what was built, what works, what's broken, what's next.

3: The Interrogation System

Before you even write your documentation, make AI tear your idea apart.

This is the prompt that changes everything:

"Before writing any code, endlessly interrogate my idea in Planning mode only. Assume nothing. Ask questions until there's no assumptions left."

AI hallucinates when your clarity ends. So if you extend your clarity, you'll force AI to find the gaps in your thinking before it starts building on a broken foundation.

What AI should ask you:

Who is this for? What's the core action a user takes? What happens when they complete that action? What data needs to be saved? What data needs to be displayed? What happens on error? What happens on success? Does this need a login? Does this need a database? Does this need to work on mobile?

If you can't answer these questions, you're not ready to build. Go answer them first.

Here's what good interrogation looks like. Say you're building a recipe-sharing app:

Who is this for? Home cooks who want to save and share recipes. Core action? User creates a recipe with a title, ingredients list, steps, and a photo. What happens after? Recipe is saved to their profile and visible on a public feed. Data saved? Recipe title, ingredients (array), steps (array), photo URL, author ID, timestamp. Data displayed? Feed of recent recipes, individual recipe detail page, user's own recipe collection. Error states? Upload fails, missing required fields, unauthorized edit attempt. Login required? Yes, to create recipes. Browsing is public. Database? Yes, users table and recipes table with a foreign key relationship. Mobile? Yes, most users will add recipes from their phone while cooking.

Those answers aren't just answers. They're the raw material for your canonical markdown docs. The user description feeds your PRD. The data structure feeds your BACKEND STRUCTURE. The flows feed your APP FLOW. The mobile requirement feeds your FRONTEND GUIDELINES.

Once the interrogation is complete and you have clear answers to everything, use this second prompt:

"Based on our interrogation, generate my canonical documentation files: -

[PRD.md](#)

- APP_FLOW.md - TECH_STACK.md - FRONTEND_GUIDELINES.md - BACKEND_STRUCTURE.md - IMPLEMENTATION_PLAN.md. Use the answers from our conversation as the source material. Be specific and exhaustive. No ambiguity."

AI will draft all docs from the interrogation output. You review them, correct anything vague, add anything missing, and lock them as your source of truth.

The sequence: Interrogation → Documentation → Code. And never, ever skip these steps.

4: UI vs UX

Two terms everyone throws around, but nobody explains them simply.

UI is what it looks like (colors, fonts, button shapes, spacing), i.e. the visual layer. **UX** is how it feels to use (Can someone figure out what to do? Is the flow intuitive? Do people get stuck? Are they frustrated or delighted?)

You can have beautiful UI and terrible UX. Pretty buttons that nobody knows how to click.

You can have ugly UI and great UX. Functional, clear, and people know exactly what to do but it's hideous.

When you talk to AI, be specific about which one you mean. "Make it look better" is UI. "Make it easier to use" is UX. "Make the button more obvious" is both.

Here's a move most people don't know: use screenshots as references. Find an app or site you love (maybe on Framer or Dribbble), screenshot it, and feed it directly to Claude Code or Cursor. Prompt "match this layout", "use this exact UI reproduced", or "use this UI as inspiration". AI can analyze images and extract design patterns, spacing, color palettes, typography, and component structures from a screenshot. This is infinitely better than trying to describe a design with words. Gemini 3 Pro from Google is currently the best model I've found at designing matching premium aesthetics.

You can also screenshot your own work in progress and say "here's what it looks like now, here's what's wrong, fix it." Visual references eliminate ambiguity faster than any written description.

Even better: Cursor has a browser sidebar that lets you design and code simultaneously. You can see your app rendered live, click on elements, move things around, update colors, test layouts, and experiment with CSS in real time, then apply those changes directly to your codebase through Agent mode. This is the fastest way to iterate on visual design without the screenshot, paste, and wait loop.

Design styles you need to know:

These are the visual languages dominating right now. When you reference them in your FRONTEND GUIDELINES .md or in prompts to AI, these terms unlock specific, recognizable aesthetics instead of vague descriptions.

Glassmorphism - frosted glass effect. Translucent elements with background blur, subtle borders, and soft shadows floating over colorful backgrounds. Think Apple's macOS, Windows 11, Spotify's mobile app. Use backdrop-filter: blur() in CSS. Creates depth and hierarchy without heavy shadows. Looks premium. Works great for cards, modals, navigation bars, and dashboards. The risk: transparency can reduce readability, so keep text contrast high.

Neobrutalism - raw, bold, intentionally unpolished. High-contrast colors, thick black borders, flat shadows, clashing palettes, quirky fonts. Think Gumroad, early Notion vibes. It's minimalism with attitude. Works well for creative brands, portfolios, indie tools. Stands out because everything else looks the same. Tell AI: "neobrutalist style with thick borders and bold primary colors."

Neumorphism (Soft UI) - elements look like they're extruded from or pressed into the background. Soft, diffused shadows on both sides create a tactile, 3D feel. Subtle and elegant but tricky for accessibility because low contrast can make buttons hard to distinguish. Works best for small UI elements, toggles, sliders, cards. Not great as your entire design language.

Bento grid - modular layouts where content is arranged in blocks of different sizes, like a Japanese bento box. Apple popularized this. Cards of varying dimensions create visual rhythm and hierarchy. Big cards for important content, smaller cards for secondary information. Responsive by nature because the grid rearranges on mobile. Perfect for dashboards, product pages, feature showcases, portfolios. This is probably the most practical trend to learn because it solves real layout problems.

Dark mode - not just a preference toggle anymore, it's a design system. Dark backgrounds with light text, careful contrast ratios, and muted accent colors. Reduces eye strain, saves battery on OLED screens, and looks premium. If you're building any consumer app, plan for both light and dark mode from the start. Don't add it on randomly later. Define both palettes and themes in your `FRONTEND_GUIDELINES.md`.

Kinetic typography - text that moves, stretches, reacts to scroll or cursor. Headlines that animate on entry, text that scales as you scroll, and interactive type treatments. Not just fade-ins. With modern CSS and libraries like Framer Motion, this is achievable without heavy JavaScript. Use sparingly for hero sections and key moments when applicable design wise.

Micro-interactions - small animations that respond to user actions. A button that subtly scales on hover. A checkbox that bounces when clicked. A loading spinner that feels alive. These tiny details separate polished products from truly amateur ones. They communicate that the interface is responsive and alive and premium. Framer Motion and CSS transitions handle these easily.

When you're prompting AI, don't say "make it look modern." Say "glassmorphism cards with bento grid layout, dark mode, and micro-interactions on hover states." That's a specific, buildable direction.

Lock your design decisions in `FRONTEND_GUIDELINES.md` before coding starts. Pick one or two of these styles, define your color palette, spacing scale, border radius, shadow values, and animation timing, and AI will follow them consistently if they're documented solidly. If they're not documented, every component will look different and there will be ZERO consistency.

5: Components

A component is a reusable piece of interface.

Think legos. Each brick is a component. You add them together to build something.

A button is a component. A navigation bar is a component. A card showing a product is a component. A form is a component made of smaller components (inputs, labels, buttons).

Why this matters for vibe coding:

When you say "build me a landing page," AI has to decide what components to create. If you don't specify, it guesses. And it might build everything as one giant mess instead of clean, reusable pieces.

Better prompt:

"Build a landing page with these components: navbar, hero section, features grid (3 cards), testimonial carousel, CTA section, footer."

Now AI knows exactly what pieces to create. Each piece is isolated. Each piece can be edited independently. That's the power of thinking in components.

6: Layout

Layout is where things go on the page.

Every website is boxes inside boxes. That's the concept. Master this... You'll understand 90% of web design.

The main boxes: header/navbar at the top, main content in the middle, optional sidebar beside it, footer at the bottom.

Inside main content, more boxes: sections that divide the page, containers that control width, grids that organize items into rows and columns, cards that group related content.

When you describe layout to AI:

"Two-column layout. Sidebar on the left, 250px wide. Main content takes the rest. Sidebar is fixed, doesn't scroll."

That's complete layout instruction and now AI knows exactly what to build.

7: State

State is data that changes.

When you click a button and something happens, state changed. When you type in a form and see your text appear, state changed. When you toggle dark mode and colors flip, state changed.

State is why your app feels alive. Without state, everything is static. Nothing responds as it should.

Common state examples: Is the menu open or closed? Is the user logged in or logged out? What items are in the shopping cart? What text is in the input field? Is this loading or done loading? Did this succeed or fail?

When your button doesn't do anything, it's usually a state problem. The click happened, but nothing told the app to update.

When you talk to AI about interactivity:

"When the user clicks this button, set the modal state to open. When they click outside the modal, set it to closed."

Now AI knows what state to track and when to change it.

8: Styling

Styling is how you make things look good.

CSS is the language that controls appearance. Colors, spacing, fonts, sizes, positions.

Tailwind is a shortcut system. Instead of writing CSS files, you add classes directly to elements. `bg-blue-500` makes background blue. `text-xl` makes text bigger. `p-4` adds padding.

Design tokens are consistent values you can reuse. Your brand blue isn't just any blue, it's the same hex code everywhere. Your spacing isn't random, it's always multiples of 4px. Your border radius is the same on every card. Your shadow values are identical on every elevated element. Using design tokens help me minimize hardcoded colors and UI hallucinations.

This is what separates amateur apps from polished ones. When every component uses the same design tokens, your app feels cohesive. When each component invents its own values, it looks like five different people built it.

Lock all of this in your `FRONTEND GUIDELINES.md` before you start coding:

Your color palette with exact hex codes for primary, secondary, background, surface, text, borders, success, error, and warning. Your spacing scale (4px, 8px, 12px, 16px, 24px, 32px, 48px, 64px). Your font stack with sizes for headings, body, and small text. Your border radius values. Your shadow definitions. Your transition timing.

When AI has this doc, every component it generates matches. Without it, you'll spend hours manually fixing inconsistencies that shouldn't exist. And trust me, I've been there, and it's a b*tch. Avoid the pain.

The more specific your styling instructions, the less AI guesses. "Make the background [#3B82F6](#)

with 16px padding and 8px border radius" beats "make it blue with some padding" every time.

9: Responsive Design

Responsive means your site works on all screen sizes.

Your laptop is wide. Your phone is narrow. Same site, different layouts.

On big screens, show more columns. On small screens, stack things vertically. On big screens, show full navigation. On small screens, show a hamburger menu.

Breakpoints are the screen widths where the design changes. Mobile is 0-640px. Tablet is 640-1024px. Desktop is 1024px and up.

Mobile-first means you design for the smallest screen first, then add complexity for larger screens. This is how Tailwind works by default. Styles without a prefix apply to all screens. Prefixed styles like md: and lg: only kick in at those breakpoints. So flex flex-col md:flex-row means stacked on mobile, side by side on tablet and up.

This isn't a preference, it's a strategy. Building mobile-first forces you to prioritize content aesthetic and minimalism. What absolutely must be on the small screen? That's your core. Everything else is progressive enhancement for bigger screens. And when you're restrained to a smaller size like this, it also pushes you to ensure the user experience is easy, minimalistic, and truly sexy.

Define your breakpoints and responsive patterns in FRONTEND_GUIDELINES.md.

"Navigation: hamburger menu below 768px, full horizontal nav above. Grid: single column mobile, two columns tablet, three columns desktop. Font sizes: scale up 15% at each breakpoint." When AI has these rules documented, every component it builds responds correctly to every screen size without you repeating the instructions.

Always think about mobile first. 50%+ of your users are likely on phones. If you only test on your laptop, you'll ship a broken experience for most people.

10: Pages vs Routes

A page is what the user sees. A route is the URL that shows that page.

yoursite.com/

shows the home page.

yoursite.com/about

shows the about page.

yoursite.com/products/123

shows product 123.

Routes can be static (/about always shows the same page) or dynamic (/products/[id] shows different products based on the id).

When you describe your app to AI:

"I need 4 pages: home (/), about (/about), all products (/products), single product (/products/[id])"

Now AI knows the full structure. It builds navigation that links correctly and creates the right files in the right places.

11: Frontend vs Backend

Frontend is what users see and interact with. The interface that runs in their browser.

Backend is what happens behind the scenes. The databases, user accounts. processing, and what runs on a server.

When you submit a form: frontend collects your input, sends it to the server, server validates and saves it, server sends back confirmation, frontend shows success message.

For simple sites you might not need a backend. Static pages, no database, no accounts. For apps with users, saved data, or complex logic, you need both.

Tell AI which one you need upfront. "This is frontend only, just a landing page" and "This needs a backend with user accounts and a database" produce completely different outputs.

12: APIs

An API is how two systems talk to each other.

Your frontend needs data from your backend. It asks via an API. So if your app needs weather data, it asks a weather service via their API. If your app needs to process payments, it talks to Stripe via their API.

Think of an API as a waiter. You tell the waiter what you want. The waiter goes to the kitchen and brings back your order.

Common patterns: GET retrieves data, POST sends data, PUT updates data, DELETE removes data.

When you talk to AI:

"When the page loads, make a GET request to /api/products and display the results in a grid."

Now AI knows exactly what to build.

13: Databases

A database is where you save stuff permanently. Without a database, everything resets when you refresh. User signs up, refresh, gone. Item added to cart, refresh, gone.

Common options for beginners:

Supabase is the easiest start. Free tier, good docs, and handles auth too. If you're not sure what to use, use Supabase.

When do you need a database? Users can create accounts. Users can save their own content. You need to store submissions. You have data that grows over time.

When do you NOT need one? Static site, same content for everyone. Portfolio with no user interaction. Landing page that just collects emails through a form service.

When talking to AI:

"Use Supabase. I need a users table with email and password, and a posts table with title, content, and user_id."

Now AI knows your data structure and can build everything to match.

14: Authentication

Authentication is login/logout AKA proving who someone is.

This is harder than it looks. Passwords need to be encrypted, sessions need to be managed, and tokens need to be handled. It's easy to mess up.

Don't build auth from scratch. Use a service. **Clerk** is the simplest with great UI out of the box. **Supabase Auth** works if you're already using Supabase for your database.

When talking to AI as an example:

"Use Clerk for authentication. Users can sign up with email or Google. After login, redirect to the dashboard."

Let the service handle the hard parts so you can just plug it in.

15: File Types

You'll see a lot of files in your codebase. Here's what some of those actually are:

.html is the structure of a webpage. .css is styling rules. .js is JavaScript that makes things interactive. .jsx is JavaScript with HTML-like syntax for React. .ts is TypeScript, which is JavaScript with types that catch errors. .tsx is TypeScript with React. .json is structured data. .md is markdown, formatted text. .env is environment variables and secrets. .gitignore tells git which files to skip.

The important one: **.env**. This is where you put API keys, secrets, passwords. Never share this file. Never commit it to git. Never screenshot it. If you leak your .env, you leak access to everything and some lucky degenerate is now going to run up your API bill for thousands of dollars. Don't make that mistake.

16: Folder Structure

Where your files go matters. A messy project confuses AI.

Standard structure:

my-app/ ├── src/ | ├── app/ → pages and routes | ├── components/ → reusable UI pieces | ├── lib/ → utilities, helpers | └── styles/ → CSS files ├── public/ → images, static files ├── .env → secrets (never share) ───

[CLAUDE.md](#)

→ AI rules and context ├── progress.txt → session tracking ───

[PRD.md](#)

→ product requirements ├── APP_FLOW.md → user flows and navigation ├── TECH_STACK.md → locked dependencies ├── FRONTEND_GUIDELINES.md → design system ├── BACKEND_STRUCTURE.md → database and API spec ├── IMPLEMENTATION_PLAN.md → step-by-step build order ├── package.json → dependencies ───

[README.md](#)

→ project overview

When AI generates code, tell it where to put the code. "Create the button component in src/components/Button.tsx." If you don't specify, AI might put your files anywhere and nothing may connect properly.

17: Your Documentation System in Practice

Markdown is not optional. It's the language AI thinks in.

Every .md file you write becomes a reference document AI can read, understand, and follow. This is why the documentation first approach works. You're not writing docs for humans. You're writing constraints for AI.

Here's how each canonical markdown doc gets used during a build:

[PRD.md](#)

is your source of truth for scope. When AI tries to add a feature you didn't ask for, you point it back to the PRD. "Only build what's in

[PRD.md](#)

." When you're deciding what to build next, you check the PRD. It kills scope creep before it even starts.

APP_FLOW.md is what you reference when building page transitions and user journeys. "Build the login flow exactly as documented in APP_FLOW.md section 3." AI now knows every screen, every redirect, and every error state in that flow.

TECH_STACK.md gets referenced at project initialization and whenever AI tries to introduce a new dependency. "Use only the packages listed in TECH_STACK.md. Do not add new dependencies without asking." This prevents AI from randomly importing libraries you didn't approve.

FRONTEND_GUIDELINES.md is the doc AI should reference for every component it creates. Your colors, spacing, typography, component patterns, responsive rules. "Style this component per FRONTEND_GUIDELINES.md section 2." Consistent design across every file.

BACKEND_STRUCTURE.md defines your data layer. "Create the users table as defined in BACKEND_STRUCTURE.md section 2.1." AI builds exactly the schema you specified, not its own interpretation.

IMPLEMENTATION_PLAN.md is your execution sequence. "We're on step 4.2 of IMPLEMENTATION_PLAN.md. Build this step only." This prevents AI from jumping ahead or building things out of order.

[CLAUDE.md](#)

is the master configuration file. It sits in your project root and Claude Code reads it automatically every session. It contains the condensed rules from all six docs: tech stack summary, naming conventions, file structure, component patterns, forbidden actions. Think of it as the operating system manual AI loads in for context before doing anything.

Here's the move most people miss:

[CLAUDE.md](#)

is a living document. Every time AI makes a mistake and you correct it, end with: "Edit

[CLAUDE.md](#)

so you don't make that mistake again." Claude is eerily good at writing rules for itself. Over time, your

[CLAUDE.md](#)

becomes a self improving rulebook. The mistake rate drops measurably because AI is literally encoding its own corrections along the way (ideally what you want to be happening).

Take this further with a

[lessons.md](#)

file. After every correction, after every PR, after every debugging session, Claude updates

[lessons.md](#)

with the pattern that caused the issue and the rule that prevents it. Point your

[CLAUDE.md](#)

at it: "Review

[lessons.md](#)

at session start for relevant project." Now AI learns from its own history on your project.

This is the self improvement loop that separates a good

[CLAUDE.md](#)

from a great one.

Example

[CLAUDE.md](#)

:

Tech stack: Next.js 14, TypeScript, Tailwind CSS, Supabase All components go in src/components/ Use functional components with hooks All API routes go in src/app/api/ Never use inline styles. Always use Tailwind. Design tokens: primary blue

[#3B82F6](#)

, background

[#F9FAFB](#)

Mobile-first responsive approach Reference docs:

[PRD.md](#)

, APP_FLOW.md, TECH_STACK.md, FRONTEND_GUIDELINES.md, BACKEND_STRUCTURE.md, IMPLEMENTATION_PLAN.md Read progress.txt at the start of every session. Update progress.txt after completing any feature. Review

[lessons.md](#)

at session start. Update it after every correction.

When Claude Code reads this, it follows these rules across every file it creates with no drift and no random decisions. Pure consistency.

Cursor has its own version of this: .cursor/rules files. Same concept, different tool. Put your project rules in .cursor/rules/ and Cursor reads them automatically across all modes. If you're using both Cursor and Claude Code on the same project, keep

[CLAUDE.md](#)

and your Cursor rules aligned. Same constraints, same conventions, and same source of truth. Some people maintain one and copy to the other. Some write a shared rules file and reference it from both. Either works. The point is that every AI tool touching your codebase follows the same rules.

progress.txt is your session bridge. Update it after every feature:

COMPLETED: User auth via Clerk (login, signup, Google OAuth) Dashboard layout with sidebar navigation Products API endpoint (GET /api/products) IN PROGRESS: Product detail page (/products/[id]) Need to connect frontend to API NEXT: Shopping cart functionality Checkout with Stripe KNOWN BUGS: Mobile nav doesn't close after clicking a link

When you open a new terminal, start a new Claude Code session, switch branches, or come back after a week, AI reads this and knows exactly where you are. No "what were we working on?" No rebuilding context all over again from scratch. No re-explaining your entire project yet again the 69th time.

This is the system. Canonical docs define what to build.

[CLAUDE.md](#)

enforces the rules. Progress.txt preserves state between sessions. Together they give AI everything it needs to build your project without hallucinating.

The more markdown documentation you have, the less AI guesses. The less AI guesses, the better off you'll be.

18: The Tools and How to Use Them

You don't need 50 tools. You need these. But you need to use these correctly. Different tools for different phases of the build.

Cursor - your code editor. AI-native, built for this workflow. It sees your entire project, understands file relationships, and can edit across multiple files at once. But Cursor isn't just one thing. It has four modes and most people only use one:

Ask mode is read-only. AI reads your codebase and answers questions without changing anything. Use it when you're exploring unfamiliar code, trying to understand how something works, or planning your next move before touching files. Think of it as your code consultant. "What does this function do?" "Why is this component re-rendering?" "What would break if I changed this schema?" Ask mode is where you think before you act.

Plan mode is where you architect before you code. You describe what you want to build and Cursor creates a detailed implementation plan with steps, asks you clarifying questions, and can generate visual diagrams of the approach. Use it at the start of every new feature. "I need to add Stripe checkout to this app. Create a plan." Review the plan, adjust it, approve the steps. Then send those steps to Agent mode to execute.

Agent mode is the workhorse. This is where AI autonomously writes code, edits files, runs terminal commands, installs packages, and fixes errors. It reads your entire project, follows your rules files, and implements features end to end. When you say "build the dashboard page," this is the mode doing the work. Most vibe coding happens here.

Debug mode is the one people don't know about. When you hit a stubborn bug, Debug mode doesn't just try random fixes. It instruments your code with runtime logs, generates multiple hypotheses about what's wrong, asks you to reproduce the bug, tests its fix, and asks you to verify. It's a structured debugging loop built into the editor.

The workflow inside Cursor: Ask to understand → Plan to architect → Agent to build → Debug when it breaks.

Claude - whether through

[Claude.ai](https://claude.ai)

or Claude Code. Claude is your thinking partner. Use Claude for the heavy thinking: interrogating your idea, writing your six canonical docs, planning architecture, working through complex product decisions, and drafting your

[CLAUDE.md](#)

. Claude is where you do the asking and planning and writing that feeds everything else later.

Claude Code runs in your terminal and automatically reads

[CLAUDE.md](#)

from your project root. It follows your rules without you repeating them. For larger refactors, documentation heavy tasks, and multi file architectural changes, Claude Code is your best tool.

Kimi K2.5 - the open-source visual coding model from Moonshot AI. This is the specialist for frontend implementation. K2.5 is a native multimodal model, meaning it was trained on vision and text together from the ground up. You can feed it a screenshot, a video, or a design mockup and it generates functional frontend code that closely matches the visual. Layouts, animations, interactions, responsive behavior. Where other models approximate your design, K2.5 replicates it. Use Kimi K2.5 (through Kimi Code or in Cursor with the model selector) when you're translating designs into code. *You have your FRONTEND_GUIDELINES.md defining the system, you have your screenshots for reference, and you need pixel-accurate implementation. That's K2.5's lane.*

Codex - OpenAI's coding agent. This is your debugger and finalizer. Codex runs in the cloud, in your terminal via Codex CLI, or directly in your IDE. Each task gets its own isolated sandbox preloaded with your repo. What makes Codex different is how it approaches debugging: it reads your codebase, traces dependencies, reviews configuration, and proposes fixes across files. It can run your tests, fix failures, and iterate until everything passes.

Use Codex after your files and architecture are built. When the structure is in place but things are breaking, when you need code review before shipping, when you want to refactor without breaking existing functionality, when you need someone to find the bugs you missed. Codex is the closer. You can also run multiple Codex tasks in parallel on different bugs or features. It works asynchronously. Start tasks, go do something else, come back to review the results.

The multi-tool workflow:

Claude does the thinking. Claude writes your docs, plans your architecture, and makes product decisions. This is where you ask/plan/think.

Cursor Agent mode (or Claude Code, or Kimi K2.5) does the building. It will implement features from the plan, generate components, connect frontend to backend. Choose the

model you leverage based on the task: - K2.5 for visual-heavy frontend work - Claude Code for architecture and documentation-heavy work - Cursor Agent for general implementation

Codex does the debugging and finishing. Run it against your built codebase. Let it find bugs, trace failures, review your code, and propose fixes. Let it run tests until they pass. And then, ship clean.

GitHub - this is where your code lives in the cloud with version control and every change tracked. You can go back to any previous version. If you break something, you can undo it. Non-negotiable for your stack.

Vercel - this is for deployment. Push to GitHub, Vercel automatically builds and deploys your code. You get a live URL and the free tier on Vercel is generous. Your app goes from your computer to live on the internet in a matter of minutes.

Supabase - this is for database and auth. They have a free tier and it's easy to setup. Handles the backend stuff so you can focus on the product.

Master these tools. Understand when to use each one. The difference between someone who ships and someone who's stuck is not which AI model they use is knowing which AI model to use when applicable.

18.5: Advanced Workflow (When You're Ready)

Once you've built a few projects with this basic system, these are the techniques that will multiply your speed. These come directly from the people who build Claude Code for a living.

Parallel sessions with git worktrees. This is the single biggest productivity unlock. Instead of working on one thing at a time, spin up 3 to 5 git worktrees, each running its own Claude Code session in parallel. One worktree builds the auth system. Another builds the dashboard layout. A third handles the API endpoints. They all share the same repo but work independently. You bounce between them, review outputs, approve changes, and merge when each is done. What used to take a full day of sequential building becomes a few hours of parallel execution. Name your worktrees and set up shell aliases so you can hop between them in one keystroke.

Plan mode as your force multiplier. You already know to use Cursor's Plan mode. Here's how to make it even more powerful: have one Claude session write the plan, then spin up a second session and tell it to review the plan as a staff engineer. "Review this implementation plan. Find gaps, edge cases I missed, and anything that will break." Fix the plan before writing a single line of code. When something goes sideways during implementation, stop immediately. Don't keep pushing. Switch back to plan mode and re-

plan from where you are. Use plan mode for verification steps too, not just for building. "Plan how to verify that this auth flow handles all edge cases."

Subagents for complex problems. When a problem is big enough that one Claude session would choke on the context, use subagents. Append "use subagents" to your request and Claude will spin up focused sub-sessions for research, exploration, and parallel analysis while keeping your main session's context window clean. One task per subagent. Think of it as delegating to a team instead of doing everything yourself.

Custom skills and slash commands. If you do something more than once a day, turn it into a reusable skill or slash command and commit it to git. Build a /techdebt command that runs at the end of every session to find and kill duplicated code. Build a context-sync command that pulls the last week of Slack, docs, and GitHub activity into one dump so Claude starts every session fully informed. Skills are how you teach Claude new capabilities specific to your workflow. They compound across every project you touch.

Autonomous bug fixing. Stop hand-holding the debugger. When you get a bug report, paste it into Claude and say "fix." When CI tests fail, say "Go fix the failing CI tests." Don't explain how. Point Claude at logs, error traces, and failing tests. Let it trace the problem, find the root cause, and resolve it. Zero context switching from you. Claude is surprisingly capable at reading docker logs, tracing distributed systems, and fixing things you'd spend hours on manually.

Voice dictation for better prompts. You speak three times faster than you type, and your prompts get dramatically more detailed as a result. On macOS, hit the fn key twice to start dictating. Describe what you want conversationally, with all the nuance and context you'd normally skip because typing is slow. Longer, more detailed prompts produce better outputs. Voice is the shortcut to getting there.

Learning mode. When you want to understand what AI is doing, not just let it do it, enable the "Explanatory" or "Learning" output style in Claude Code's config. Claude will explain the reasoning behind every change it makes. You can also ask Claude to generate visual HTML presentations explaining unfamiliar code, draw ASCII diagrams of architectures and protocols, or build spaced-repetition flashcards from new concepts. The people who learn while they build are the ones who eventually stop needing this guide.

19: Git and Version Control

Git tracks every change you make.

Without git: you break something and can't undo it, you lose track of what changed, one mistake can destroy everything.

With git: every change is saved, you can go back to any previous version, your code lives on GitHub not just your computer.

The basics:

`git add .` stages your changes. `git commit -m "message"` saves them with a description. `git push` uploads to GitHub. `git pull` downloads latest changes.

When you're vibe coding, commit often. After every major feature that works: "Added user login" "Added product grid" "Fixed checkout bug." If you break something, you can always go back.

This connects to your progress.txt system. Commit code, update progress.txt, push both. Now GitHub has your code and your context file. Next session, pull down, read progress.txt, keep building.

20: Environment Variables and Secrets

Your API keys, database passwords, and secrets should never be in your public or production codebase.

They go in a .env file. Then in your code. Access them through `process.env.YOUR_KEY_NAME`.

Rules: Never commit .env to git (add it to .gitignore). Never paste API keys in chat or screenshots with AI apps. If you do that, assume it's compromised. Never put secrets in frontend code, only backend. Use different keys for development and production.

When you deploy to Vercel, add your environment variables in Vercel's dashboard settings. They won't transfer from your local .env automatically.

If you leak a key, immediately go to that service and revoke it. Create a new one.

21: Deployment

Your code works on your computer. Now it needs to work on the internet.

The process with Vercel:

1. Push code to GitHub
2. Connect GitHub repo to Vercel
3. Vercel automatically builds and deploys
4. You get a URL like

your-app.vercel.app

5. Add your environment variables in Vercel's dashboard

When something works locally but breaks when deployed, give AI the Vercel error logs. "It works on localhost but breaks on Vercel. Here's the error from Vercel's logs:" and paste the error.

99% of deploy bugs are missing ENV variables or wrong build settings.

22: Reading Error Messages

Errors are not insults, they're instructions.

An error message tells you exactly what went wrong. Most people panic and ignore them. Please don't do that.

Anatomy of an error:

TypeError: Cannot read property 'map' of undefined at ProductList
(src/components/ProductList.tsx:15:23)

Translation: TypeError means you're using something wrong. "Cannot read property 'map' of undefined" means you tried to use `.map()` on something that doesn't exist.

ProductList.tsx:15:23 tells you the exact file and line number.

When you get an error, give AI everything:

"I'm getting this error: [full error message]. Here's the code at that line: [paste the relevant code]"

The more context you give AI for the error, the faster the potential fix for your problems.

23: The Debugging Loop

When something breaks:

1. Read the error. Actually read it.
2. Find the location. What file, what line.
3. Understand the claim. What does the error say is wrong?
4. Check the obvious. Typos, missing imports, wrong variable names.
5. Give AI context. Error + code + what you expected.

The loop: AI gives you code → you try it → it breaks → you paste error → AI fixes → repeat until it works.

For stubborn bugs that survive two or three rounds of this loop, switch tools. Use Cursor's Debug mode. It generates multiple hypotheses, instruments your code with runtime logs, and walks through the bug methodically instead of guessing. Or fire up Codex. Give it the bug description and let it trace through your codebase, identify the root cause across files, and iterate until tests pass. Codex is especially good at bugs that span multiple files or involve data flow issues, because it reads the entire repo before proposing a fix.

This is normal. Vibe coding isn't generally meant to be a one-shot approach even though we love to see it that way. It's iteration. The skill is iterating quickly and knowing which tool to use when, not avoiding iteration entirely.

24: Verifying Before You Ship

Before you ship, check:

Does it work on mobile? Actually open it on your phone. Does it work in different browsers? What happens with no data, are empty states handled? What happens with wrong data, are error states handled? What happens with slow internet, do loading states exist? Can you break it by clicking fast? Are secrets hidden from browser dev tools?

Don't ship until you've answered these. Your users will find every bug you missed. Do this process meticulously.

25: Talking to AI About All of This

Now that you have the vocabulary... You need to use it.

Vague prompt:

"Build me an app where users can post things"

Specific prompt backed by documentation:

"Read

[CLAUDE.md](#)

and progress.txt first. Then build step 4.2 from IMPLEMENTATION_PLAN.md. The login flow is defined in APP_FLOW.md section 3. Use the auth setup from BACKEND_STRUCTURE.md section 5. Style everything per FRONTEND_GUIDELINES.md. Match the UI to the screenshot attached."

Same idea. A completely different output quality.

The specificity isn't extra work, it's "the work". The more you define upfront, the less you debug later.

26: How to Read AI's Output

AI made you some code. Do you know what you're looking at?

You don't need to understand every line. But you need to understand the structure. What files were created? What do they do? How do they connect?

When AI generates code, ask this: "Explain what you just built in plain English. What does each file do? How do they connect?"

Over time, you'll start recognizing patterns. You'll see an import statement and know it's pulling in another file. You'll see a useState and know it's tracking something that changes. You'll see an API call and know it's fetching data.

This is how you go from vibe coder to actual builder. Not by memorizing syntax, but by understanding patterns.

27: How to Iterate

Everyone's first output is rarely right... And that's fine.

The iteration system:

1. AI builds version 1
2. You test it, find what's wrong
3. You describe what's wrong specifically (not "it's broken," but "the submit button doesn't save to the database, here's the error")
4. AI fixes it
5. You test again
6. Repeat

Good iteration: "The product grid shows 4 columns on desktop but I need 3. The card images are stretched, they should be object-cover. And there's no loading state when data is fetching."

Bad iteration: "It doesn't look right, fix it."

Be specific. Always.

28: Breaking Big Ideas into Small Pieces

AI chokes on larger, lofty requests. "Build me a full e-commerce site" will likely produce absolute garbage.

Break it down into pieces:

1. Set up project scaffold and install dependencies
2. Build the navbar component
3. Build the product card component
4. Build the product grid page
5. Connect to database and fetch products
6. Build the single product page
7. Add to cart functionality
8. Build the cart page
9. Checkout with Stripe

Each piece is one conversation or one task. Each builds on the last and each can be tested independently. Take yourself back to the LEGO days. This is actually that.

This is literally what your IMPLEMENTATION_PLAN.md is. That list above, numbered and sequenced, is the exact format of your implementation plan. If you did the work in Part 2, you already have this breakdown written for you. You don't create it on the fly while coding. You created it before you started. Now you're just executing it. Step by step with a plan.

Tell AI: "Build step 5 from IMPLEMENTATION_PLAN.md." Not "build the next thing."
Precision compounds.

This connects back to your progress.txt file. After each piece, update progress. Start next piece with fresh context.

29: When AI Is the Wrong Tool

Sometimes you just need to learn the thing.

Use AI for generating boilerplate code, writing repetitive logic, exploring approaches quickly, debugging with context, and translating your intent into code.

Learn yourself: core concepts (everything in this guide), how to read code AI generates, how to spot when AI is wrong, how to debug when AI can't help, and how your chosen stack works fundamentally.

If you rely on AI for everything, you're building on quicksand. One weird bug and you'll be stuck, and sinking. If AI explains something wrong and you believe it, well... you're fucked. So maybe, just maybe learn these things now.

Also: a lot of the times website docs are better than AI info. Stack Overflow has exact answers to exact errors. Documentation sites have authoritative information. AI is great for synthesis and generation, but when it fails, knowing how to search official docs and sites is the fallback skill nobody talks about. And Claude can even teach itself everything it needs from these.

30: Scope and Knowing When to Stop

A never ending feature list kills more projects than some bad code.

You're done when the core feature works, users can complete the main action, it doesn't break on common paths, and it's deployed and accessible.

You're NOT done when it's "perfect," every edge case is handled, it has every feature you imagined, or it looks exactly like Dribbble designs.

Ship the simple version and get your feedback. Then, improve based on real usage.

The best app you never ship is worse than the average app that's live.

31: Maintaining What You Built

You built it and now you'll need to change it:

Future you (or AI) needs to understand the code. This is where your documentation system really pays off.

Your README explains what the project is. Your

[CLAUDE.md](#)

enforces the rules. Your progress.txt shows what was built and what's next. Your canonical markdown docs define every aspect of the product, from requirements to implementation sequence.

When you return to code after 3 months:

"Read

[CLAUDE.md](#)

, progress.txt, and

[PRD.md](#)

. I'm returning to this project after a break. Summarize where things stand against the IMPLEMENTATION_PLAN.md and what needs attention."

Good documentation makes future vibe coding sessions 10x faster. Bad documentation means starting over from scratch.

Keep dependencies updated, write comments on confusing parts, use consistent naming, and treat your codebase like a home that strangers on Airbnb will visit and need to know where things are.

32: Cost Awareness

API calls cost money. Databases cost money. Hosting can cost money. AI tools cost money.

Free tiers exist and they're generous. Vercel is free, Supabase is free, Clerk is free (up to limits), etc. But when you scale, costs may very well appear.

For the AI tools in your workflow: Cursor Pro covers most model access and is your primary subscription. Claude Code usage comes through your Anthropic or Claude plan. Codex is included with ChatGPT Plus, Pro, and higher tiers. Kimi K2.5 is open-source with free access through

[Kimi.com](#)

and generous API pricing. You don't need every paid tier on day one. Start with Cursor Pro and Claude or Gemini Pro 3. Add the others when your workflow demands it.

Know what's free vs what adds up. AI API calls (OpenAI, Anthropic) cost per token. Database storage costs past free tiers. Image hosting costs at scale.

Start free. Scale when you need to. Don't architect for millions of users on day one. Because the reality is you probably aren't hitting that at all. Start small.

33: Security Basics

The bare minimum:

Never expose API keys in frontend code. Always validate inputs on the backend (don't trust anything from the browser). Use HTTPS (Vercel does this automatically). Keep dependencies updated (outdated packages have known vulnerabilities). Use auth services instead of rolling your own.

You're not building a high security bank here, but you should also know all the basics of security so you don't ship your app with obvious vulnerabilities. And so you don't get clowned on the internet. Most people's worst nightmare.

34: Third Party Services

You don't build everything from scratch. You plug in services for the hard parts.

Auth: Clerk, Supabase Auth. Handles login, signup, sessions. **Payments:** Stripe. Handles money. **Database:** Supabase, Firebase. Handles storage. **Email:** Resend, SendGrid. Handles transactional email. **File uploads:** Uploadthing, Supabase Storage. Handles media.

When to reach for a service: whenever the feature isn't your core product. Your core product is the unique thing you're building. Everything else is infrastructure. Use existing services for infrastructure.

35: Assets, Media, and Component Libraries

Images, fonts, icons, and pre-built components. Where to get and find them, and how to use them.

Icons: Lucide React. Free, consistent, easy to use. **Fonts:** Google Fonts. Import in your layout, use everywhere. **Images:** Unsplash for stock. Your own for product shots. AI-generated for custom. **File sizes matter:** Large images slow your site. Compress before uploading. Use Next.js Image component for automatic optimization.

Component libraries are the cheat code most beginners don't know about. Instead of building every button, modal, dropdown, and form element from scratch, use a library that gives you polished, accessible, customizable components out of the box.

shadcn/ui is the dominant choice in the Next.js and Tailwind ecosystem right now. It's not a traditional library you install as a dependency. It copies component source code directly into your project, meaning you own the code and can customize everything. Buttons, dialogs, dropdowns, tabs, forms, data tables, all built on top of Radix UI primitives with Tailwind styling. Tell AI: "Use shadcn/ui components. Initialize with `npx shadcn@latest init` and add the components we need." This saves hours of building basic UI elements and gives you accessible, production-quality components from the start.

Note your component library choice in `TECH STACK .md` and `FRONTEND GUIDELINES .md` so AI uses it consistently across every page.

The Complete System

You now have everything.

Before building:

1. Run the interrogation prompt. Let AI tear brutally interrogate your idea.
2. Answer every question AI asks.
3. Use the doc generation prompt to create your six canonical docs:

[PRD.md](#)

, APP_FLOW.md, TECH_STACK.md, FRONTEND_GUIDELINES.md,
BACKEND_STRUCTURE.md, IMPLEMENTATION_PLAN.md

4. Write your

[CLAUDE.md](#)

with project rules, references to all six docs, and a

[lessons.md](#)

self-improvement loop

5. Create progress.txt with your starting state
6. Gather UI screenshots for reference
7. Initialize git and push to GitHub

While building:

1. AI reads

[CLAUDE.md](#)

, progress.txt, and

[lessons.md](#)

first, every session

2. Use Ask and Plan modes in Cursor (or Claude) to architect before coding
3. Use Agent mode, Claude Code, or Kimi K2.5 to implement (match tool to task)
4. Work in small pieces, one feature at a time
5. Give specific, vocabulary-rich prompts referencing your canonical docs

6. Use screenshot references for UI work
7. Commit to git after each working feature
8. Update progress.txt after each feature
9. After every correction, update

[CLAUDE.md](#)

and

[lessons.md](#)

so AI never makes the same mistake twice

10. Use Codex to debug, review, and finalize once architecture is in place
11. Test on mobile regularly
12. Read errors, don't panic

Before shipping:

1. Check mobile
2. Check error states and empty states
3. Verify secrets are hidden
4. Test the main user flow end-to-end
5. Check performance (is anything slow?)

After shipping:

1. Update docs to reflect what was built
2. Update dependencies periodically
3. Iterate based on real user feedback
4. Keep progress.txt and

[lessons.md](#)

current for future sessions

5. Turn repeated workflows into reusable skills and slash commands

That's the complete system.

Vibe coding isn't witchcraft black magic. All it is meticulous planning, systems, documentation, vocabulary, and iteration. You interrogate your idea. You write your markdown docs. You set up

[CLAUDE.md](#)

, progress.txt, and

[lessons.md](#)

for persistence and self-improvement. You use the right tool for each phase: Claude for asking and thinking and planning, Cursor modes for building, Kimi K2.5 for visual implementations, and Codex for debugging and finishing. You describe the work in specific terms. You track your progress between sessions. You commit your code. And you ship.

The AI now does all the typing. And you do all the thinking.

Now you have absolutely no excuses.

Go fucking build something today.